

Codex + Obsidian 模块化知识系统

核心平台接口规范

文档版本：0.1.0

状态：Draft

适用范围：Core Platform、Obsidian 插件、确定性脚本、Codex 调用层

不包含：申请、课程、实习、日记等具体业务逻辑

1. 文档目的

本文档定义核心平台对业务模块提供的公共能力、数据结构和接口契约。

核心平台的目标是：

- 在不理解具体业务领域的前提下加载和运行模块；
- 管理文件、元数据、状态、任务、审核和日志；
- 将必要上下文交给 Codex；
- 接收模块生成的结构化执行计划；
- 安全、可追踪地修改 Obsidian Vault；
- 保证模块之间低耦合；
- 允许未来新增模块而不修改核心业务代码。

本文使用以下约束词：

- **MUST**：必须满足；
 - **SHOULD**：建议满足，除非有明确理由；
 - **MAY**：可选能力。
-

2. 核心平台职责

核心平台 MUST 提供以下能力：

1. 模块发现与加载；
2. 模块实例发现与管理；
3. 分层 Inbox 扫描；
4. 输入路由；
5. 文件系统元数据提取；
6. 公共 YAML 维护；
7. 附件伴随笔记管理；
8. Codex 上下文构建；
9. 操作计划验证与执行；
10. 权限检查；
11. Review Queue；
12. Today Dashboard；

13. 调度与周期任务；
14. 模块事件总线；
15. 操作日志；
16. 状态存储；
17. Git 快照与回滚；
18. 错误恢复；
19. CLI 和插件调用接口。

核心平台 MUST NOT：

- 写死具体课程、学校、公司或申请流程；
- 决定实习日报应包含哪些业务章节；
- 决定课程笔记应关联哪些知识点；
- 自行生成领域结论；
- 允许模块绕过权限系统直接批量修改 Vault；
- 默认完整读取所有 PDF、PPT 或附件。

3. 核心概念

3.1 Module

模块定义一类通用业务能力，例如：

```
course
application-tracker
experience-log
journal
project-management
```

模块代码位于：

```
90-System/Modules/{module_id}/
```

3.2 Module Instance

实例是模块的一次具体使用。

例如：

```
模块：course
实例：COMP601-2027-S1
```

实例配置位于：

90-System/Instances/{instance_id}/

3.3 Capture

Capture 是尚未完成正式处理的输入，包括：

- Markdown 笔记；
 - 文本；
 - PDF；
 - PPT；
 - 图片；
 - ZIP；
 - 外部 AI 输出；
 - 手动投放的文件。
-

3.4 Operation Plan

模块不能直接任意修改 Vault。

模块或 Codex 应先生成结构化操作计划，由核心平台验证、授权并执行。

3.5 Review Item

Review Item 是需要用户决定的事项，例如：

- 不确定分类；
 - 不确定双向链接；
 - 修改申请状态；
 - 合并笔记；
 - 批量移动；
 - 删除文件。
-

3.6 Dashboard Item

Dashboard Item 是模块提交给 Today 页面的提示、任务、异常或进度信息。

3.7 Event

Event 是模块间交换信息的标准消息。

模块不能直接修改其他模块拥有的数据。

4. 核心目录

```
90-System/
├── Core/
│   ├── config/
│   ├── schemas/
│   └── policies/
│
├── Modules/
├── Instances/
├── Components/
│
├── State/
│   ├── module-registry.json
│   ├── instance-registry.json
│   ├── file-manifest.json
│   ├── job-state.json
│   └── event-offsets.json
│
├── Review Queue/
│   ├── Pending/
│   ├── Approved/
│   ├── Rejected/
│   └── Deferred/
│
├── Research Requests/
├── Processed Research/
├── Needs Review/
├── Logs/
└── Scripts/
```

核心平台 SHOULD 将机器状态与用户知识文档分开保存。

5. 模块发现接口

5.1 模块发现规则

核心平台 MUST 扫描：

```
90-System/Modules/*/module.yaml
```

缺少 `module.yaml` 的目录不得被加载为模块。

5.2 模块注册记录

```
{
  "module_id": "course",
  "name": "课程学习管理",
  "version": "1.0.0",
  "status": "enabled",
  "path": "90-System/Modules/course",
  "manifest_hash": "sha256:...",
  "loaded_at": "2027-02-20T09:00:00+11:00",
  "compatible": true,
  "errors": []
}
```

模块发现结果写入：

```
90-System/State/module-registry.json
```

5.3 模块状态

支持：

```
installed
enabled
disabled
error
incompatible
```

只有 `enabled` 模块可以：

- 接收 Capture；
- 注册调度任务；
- 生成 Dashboard Item；
- 订阅事件。

5.4 兼容性检查

模块 MUST 声明：

```
engine:
  min_version: 0.1.0
  max_version: 0.x
```

核心平台必须在加载前检查兼容性。

6. 实例发现接口

核心平台扫描：

```
90-System/Instances/*/instance.yaml
```

实例必须包含：

```
instance_id: COMP601-2027-S1
module: course
status: active
content_root: 20-Workspace/Courses/COMP601-2027-S1
```

实例注册记录：

```
{
  "instance_id": "COMP601-2027-S1",
  "module_id": "course",
  "status": "active",
  "content_root": "20-Workspace/Courses/COMP601-2027-S1",
  "inbox_paths": [
    "20-Workspace/Courses/COMP601-2027-S1/Inbox"
  ],
  "config_hash": "sha256:...",
  "errors": []
}
```

实例状态支持：

```
planned
active
paused
completed
archived
error
```

7. 分层输入扫描接口

核心平台必须区分以下输入级别：

```
target-directory
instance-inbox
module-inbox
global-inbox
```

优先级：

```
明确目标目录
> 实例 Inbox
> 模块 Inbox
> 全局 Inbox
```

输入上下文结构：

```
{
  "capture_id": "CAP-2027-000143",
  "path": "20-Workspace/Courses/COMP601-2027-S1/Inbox/note.md",
  "filename": "note.md",
  "extension": ".md",
  "source_level": "instance-inbox",
  "module_hint": "course",
  "instance_hint": "COMP601-2027-S1",
  "created_at": "2027-03-08T10:15:00+11:00",
  "modified_at": "2027-03-08T10:20:00+11:00",
  "file_hash": "sha256:...",
  "frontmatter": {},
  "text_preview": "今天讲了 Adam...",
  "content_read_level": 0
}
```

核心平台 MUST 在传给模块前生成该结构。

8. 文件读取接口

读取等级：

```
0 = 只读取路径和文件系统信息
1 = 读取文档属性
2 = 读取首页、标题页或目录
3 = 读取完整内容
```

请求结构：

```
{
  "file": "Assignments/A01/specification.pdf",
  "requested_level": 2,
  "reason": "extract-assignment-metadata",
  "requested_by": "course",
  "instance_id": "COMP601-2027-S1"
}
```

核心平台必须检查：

- 模块是否有权限；
- 目录策略是否允许；
- 是否已有缓存；
- 是否真的需要升级读取等级；
- Level 3 是否需要用户确认。

返回：

```
{
  "granted_level": 2,
  "cached": false,
  "content": {
    "title": "Assignment 1",
    "page_count": 8,
    "first_page_text": "..."
  }
}
```

9. 公共元数据接口

9.1 公共 Frontmatter

所有受管 Markdown SHOULD 包含：

```
---
id: NOTE-2027-0001

module: course
module_instance: COMP601-2027-S1
type: lecture-note

created: 2027-03-08T10:00:00+11:00
updated: 2027-03-08T10:30:00+11:00

processing_status: processed
```



```
review_status: not-required

authorship:
  user_written: true
  ai_enriched: true
  system_metadata: true

schema_version: 1
---
```

核心平台负责公共字段。

模块负责业务字段。

9.2 字段所有权

字段所有权分为：

```
core
module
user
external
```

核心平台 MUST 防止模块覆盖核心只读字段，例如：

- `id` ;
 - `created` ;
 - `file_hash` ;
 - `processing_run_id` 。
-

9.3 元数据更新请求

```
{
  "operation": "update-frontmatter",
  "target": "Notes/Week-03.md",
  "patch": {
    "week": 3,
    "topics": ["Adam", "Gradient Descent"]
  },
  "field_owner": "module",
  "source_module": "course"
}
```

10. 附件伴随笔记接口

核心平台提供统一组件：

```
attachment-sidecar
```

输入：

```
{
  "attachment": "Assignments/A01/assignment.pdf",
  "module": "course",
  "instance": "COMP601-2027-S1",
  "suggested_type": "assignment-specification"
}
```

输出文件：

```
assignment.pdf
assignment.md
```

伴随笔记公共字段：

```
---
id: FILE-2027-0003
module: course
module_instance: COMP601-2027-S1
type: assignment-specification

source_file: "[[assignment.pdf]]"
original_filename: assignment.pdf
file_format: pdf
file_hash: "sha256:..."

content_read_level: 0
content_indexed: false
summary_generated: false
---
```

模块 MAY 在伴随笔记中添加业务字段和正文模板。

11. 路由接口

11.1 路由请求

对于模块 Inbox 或全局 Inbox，核心平台调用模块 `match` 接口。

```
{
  "capture": {
    "capture_id": "CAP-2027-0143",
    "path": "00-Inbox/Quick Capture/note.md",
    "filename": "note.md",
    "frontmatter": {},
    "text_preview": "今天实习继续调登录接口..."
  },
  "active_context": {
    "active_instances": [
      "company-internship-2027",
      "australia-masters-2027"
    ]
  }
}
```

11.2 模块匹配响应

```
{
  "module_id": "experience-log",
  "matched": true,
  "confidence": 0.93,
  "suggested_instance": "company-internship-2027",
  "suggested_type": "daily-log",
  "reasons": [
    "正文包含实习工作记录",
    "存在活跃实习实例"
  ],
  "required_read_level": 0
}
```

11.3 路由决策

默认建议：

```
confidence >= 0.85
→ 自动路由
```

```
0.60 <= confidence < 0.85
```

→ 路由并生成审核项，或等待用户确认

```
confidence < 0.60
```

→ 放入 Needs Routing

阈值必须可以全局配置，也可以由模块覆盖。

12. 上下文包接口

核心平台调用 Codex 时，不应直接传入整个 Vault。

标准上下文包：

```
{
  "task_id": "TASK-2027-0041",
  "task_type": "process-capture",

  "global_rules": "AGENTS.md",
  "module_prompt": "Modules/course/prompts/organize-lecture.md",
  "module_manifest": "Modules/course/module.yaml",
  "instance_config": "Instances/COMP601-2027-S1/instance.yaml",
  "instance_context": "Instances/COMP601-2027-S1/context.md",

  "primary_input": [
    "COMP601-2027-S1/Inbox/note.md"
  ],

  "related_files": [
    "COMP601-2027-S1/Course.md"
  ],

  "constraints": {
    "max_related_files": 10,
    "max_read_level": 0,
    "allow_delete": false,
    "allow_external_network": false
  }
}
```

核心平台 MUST：

- 限制上下文大小；
- 只加载相关模块；
- 只加载相关实例；
- 禁止无理由扫描全 Vault；

- 记录实际读取文件清单。

13. Operation Plan 接口

Codex或模块必须返回结构化操作计划，而不是直接执行任意命令。

```
{
  "plan_id": "PLAN-2027-0033",
  "task_id": "TASK-2027-0041",
  "module": "course",
  "instance": "COMP601-2027-S1",

  "summary": "整理课堂笔记并归档到 Week 03",

  "operations": [
    {
      "operation_id": "OP-001",
      "type": "update-frontmatter",
      "target": "COMP601-2027-S1/Inbox/note.md",
      "risk": "green",
      "payload": {
        "type": "lecture-note",
        "week": 3
      }
    },
    {
      "operation_id": "OP-002",
      "type": "rename-file",
      "target": "COMP601-2027-S1/Inbox/note.md",
      "destination": "COMP601-2027-S1/Notes/2027-03-08-Week-03.md",
      "risk": "green"
    },
    {
      "operation_id": "OP-003",
      "type": "add-link",
      "target": "COMP601-2027-S1/Notes/2027-03-08-Week-03.md",
      "value": "[[Assignment 1]]",
      "risk": "yellow",
      "confidence": 0.68
    }
  ],

  "review_items": [
    {
      "operation_id": "OP-003",
      "reason": "无法确认与 Assignment 1 的关系是教师要求还是用户推测"
    }
  ]
}
```

```
]
}
```

14. 支持的操作类型

第一版 SHOULD 支持：

```
create-file
update-file
append-section
update-frontmatter
rename-file
move-file
create-directory
create-sidecar
add-link
remove-link
update-index
create-review-item
publish-event
update-instance
archive-file
```

高风险操作：

```
delete-file
overwrite-original
bulk-move
bulk-rename
merge-files
git-push
```

必须经过更严格权限检查。

15. 操作事务与原子性

一次处理运行必须拥有：

```
run_id
task_id
plan_id
```

执行流程：

```
验证计划
→ 创建 Git 快照
→ 锁定目标文件
→ 执行绿色操作
→ 创建黄色审核项
→ 更新状态
→ 写入日志
→ 释放锁
```

如果执行中失败：

- 已完成操作 SHOULD 回滚；
- 无法回滚时必须生成异常报告；
- 不得将 Capture 标记为处理完成；
- 不得丢失原始文件。

16. 权限接口

16.1 风险级别

```
green
yellow
red
```

Green

可自动执行，例如：

- 补公共 YAML；
- 创建伴随笔记；
- 明确路径下移动新文件；
- 更新索引；
- 格式化 Markdown。

Yellow

需要事后审核或等待用户决定，例如：

- 推断关联；
- 修改长期档案；
- 合并外部研究结果；
- 生成简历素材。

Red

必须事前确认，例如：

- 删除；
- 批量迁移；
- 修改日记原文；
- 覆盖官方资料；
- Git push。

16.2 权限检查请求

```
{
  "operation_type": "move-file",
  "module": "course",
  "instance": "COMP601-2027-S1",
  "target": "Notes/note.md",
  "risk": "yellow",
  "reason": "目标文件已有多个反向链接"
}
```

返回：

```
{
  "allowed": false,
  "requires_review": true,
  "review_priority": "medium"
}
```

17. Review Queue 接口

17.1 Review Item Schema

```
---
review_id: REV-2027-0042
schema_version: 1

source_module: course
module_instance: COMP601-2027-S1

target: "[[COMP601 Week 03]]"
action: add-link
proposed_value: "[[COMP601 Assignment 1]]"
```



```
confidence: medium
priority: medium
status: pending

created: 2027-03-08T11:00:00+11:00
review_after:
expires_at:
---
```

正文必须包含：

- 建议修改；
- 判断依据；
- 不确定原因；
- 用户决定；
- 执行结果。

17.2 Review 状态

```
pending
approved
approved-with-modification
rejected
deferred
resolved-by-user-edit
expired
error
```

17.3 Review 处理接口

```
{
  "review_id": "REV-2027-0042",
  "action": "approve-with-modification",
  "user_comment": "保留弱关联，但不要写入正式作业要求。"
}
```

核心平台调用模块的 `review-resolve` 工作流，并执行最终计划。

18. Today Dashboard 接口

模块通过 Dashboard Provider 提交标准项目。

```
{
  "item_id": "DASH-course-0041",
  "source_module": "course",
  "instance_id": "COMP601-2027-S1",

  "category": "review",
  "priority": "medium",
  "title": "确认课堂笔记与 Assignment 1 的关系",
  "description": "原文提到 “作业可能会用到” ",

  "target": "[[REV-2027-0042]]",
  "due_at": "2027-03-09",
  "actions": [
    "open",
    "approve",
    "defer",
    "discuss"
  ]
}
```

类别支持：

```
action
review
research
summary
deadline
warning
status
system
```

核心平台负责聚合、排序并渲染 `Today.md`。

19. 调度接口

模块可以声明周期任务。

```
scheduled_jobs:
- id: weekly-review
  scope: instance
  trigger:
    type: cron
    expression: "0 18 * * 0"
  handler: workflows/weekly-rollup.yaml

- id: due-check
```

```
scope: instance
trigger:
  type: field-due
  field: next_check
handler: workflows/create-research-request.yaml
```

核心平台记录：

```
{
  "job_id": "course:COMP601-2027-S1:weekly-review",
  "next_run": "2027-03-14T18:00:00+11:00",
  "last_run": null,
  "status": "scheduled"
}
```

电脑离线导致错过任务时，系统下次启动执行补偿检查。

20. 事件总线接口

事件结构：

```
{
  "event_id": "EVT-2027-0017",
  "event_type": "evidence.created",
  "source_module": "experience-log",
  "source_instance": "company-internship-2027",

  "entity": {
    "type": "skill-evidence",
    "id": "EVIDENCE-0021"
  },

  "payload": {
    "skill": "API Debugging",
    "source": "[[2027-03-08 实习记录]]"
  },

  "created_at": "2027-03-08T19:00:00+11:00"
}
```

核心平台第一版支持：

```
capture.created
file.added
file.moved
```

```
note.processed
metadata.updated
review.created
review.resolved
summary.generated
research.required
deadline.approaching
evidence.created
instance.archived
```

事件 MUST 持久化，避免模块离线时丢失。

21. 日志接口

每次处理生成日志：

```
---
run_id: RUN-2027-0031
task_id: TASK-2027-0041
module: course
instance: COMP601-2027-S1
status: completed
started_at: 2027-03-08T10:30:00+11:00
completed_at: 2027-03-08T10:30:12+11:00
---
```

正文记录：

- 输入文件；
- 上下文文件；
- Codex Prompt；
- 读取等级；
- 执行操作；
- 创建审核项；
- 错误；
- Git 快照；
- Token 使用量；
- 未执行操作。

22. Git 与回滚接口

核心平台 SHOULD 在以下操作前创建快照：

- Codex批量修改；
- 文件移动；

- 长期档案更新；
- YAML迁移；
- 模块升级；
- 实例归档。

快照记录：

```
{
  "snapshot_id": "SNAP-2027-0012",
  "run_id": "RUN-2027-0031",
  "git_commit": "abc1234",
  "created_at": "2027-03-08T10:29:58+11:00"
}
```

默认允许本地 commit，但 Git push 必须由用户确认。

23. CLI 接口

核心 CLI 暂定为：

```
pkb scan
pkb modules list
pkb modules validate
pkb instances list
pkb inbox scan
pkb inbox process
pkb inbox route
pkb process current
pkb review list
pkb review resolve REV-2027-0042
pkb dashboard build
pkb jobs run-due
pkb audit
pkb rollback RUN-2027-0031
```

支持限定模块和实例：

```
pkb inbox process
--module course
--instance COMP601-2027-S1
```

24. Obsidian 插件调用接口

插件不得直接实现业务逻辑。

插件只调用核心命令：

```
Quick Capture
Process Current Inbox
Organize Current Directory
Open Today Dashboard
Open Review Queue
Discuss Review Item
Generate Periodic Summary
Create Research Request
Show Recent Changes
Rollback Last Run
```

核心平台应通过本地 CLI、IPC 或 localhost 服务向插件提供接口。

25. 幂等性要求

同一 Capture 重复处理时：

- 不得重复创建文件；
- 不得重复添加链接；
- 不得重复生成相同审核项；
- 不得重复发布同一业务事件。

Operation MUST 包含幂等键：

```
{
  "idempotency_key": "course:COMP601:CAP-0143:add-parent-link"
}
```

26. 缓存要求

核心平台 SHOULD 缓存：

- 文件 Hash；
- 文档属性；
- Level 2 提取结果；
- Codex生成摘要；
- 模块匹配结果；

- Schema验证结果。

文件 Hash 未变化时，不应重复读取正文。

27. 错误模型

统一错误结构：

```
{
  "error_code": "MODULE_SCHEMA_INVALID",
  "severity": "error",
  "module": "course",
  "instance": "COMP601-2027-S1",
  "message": "lecture-note.schema.yaml 无法解析",
  "recoverable": false,
  "suggested_action": "禁用模块并检查 Schema"
}
```

错误等级：

```
info
warning
error
fatal
```

Fatal 错误不得继续执行修改。

28. 核心平台测试要求

必须覆盖：

模块发现

- 正常模块；
- 缺少 manifest；
- 不兼容版本；
- 重复 module_id。

实例发现

- 实例引用不存在模块；
- content_root 不存在；
- paused 和 archived 实例。

路由

- 明确目录；
- 实例 Inbox；
- 模块 Inbox；
- 全局 Inbox；
- 多模块冲突。

权限

- Green 自动执行；
- Yellow 创建审核；
- Red 阻止执行。

幂等

- 同一文件重复扫描；
- 同一 Operation 重复执行；
- 相同 Review Item 去重。

回滚

- 中途失败；
- 文件移动失败；
- YAML写入失败；
- Codex输出不合法。

29. 第一阶段验收标准

核心平台 MVP 满足以下条件即可验收：

1. 自动发现模块和实例；
 2. 能扫描四级输入位置；
 3. 能为 Capture 生成标准上下文；
 4. 能调用模块 `match`；
 5. 能验证并执行 Operation Plan；
 6. 能自动维护公共 YAML；
 7. 能创建附件伴随笔记；
 8. 能生成 Review Item；
 9. 能聚合 Today Dashboard；
 10. 能记录完整运行日志；
 11. 能创建 Git 快照；
 12. 重复处理不会产生重复数据；
 13. 核心代码不包含课程、申请、实习等业务判断。
-

30. 版本与兼容策略

核心接口使用语义化版本：

```
MAJOR.MINOR.PATCH
```

- MAJOR：破坏接口兼容；
- MINOR：新增向后兼容能力；
- PATCH：修复问题。

所有持久化结构必须包含：

```
schema_version: 1
```

核心平台升级前必须验证模块兼容性，并在需要时调用模块 migration。